

Задачи до понедельника:

1. Кирилл - достать результаты поиска для queries из candidates по системы SQL-HR. Для каждого запроса из queries.csv прогнать **финальную систему SQL-HR** и получить итоговый ранжированный top-15 кандидатов.

Файл:

retrieval_sql_hr.csv

Обязательные колонки

- query_id
- candidate_id
- s_system
- rankearch
- score
- latency_ms
- retrieved_at

Значение search_system = "sql_hr"

2. Амина - Для каждого запроса из queries.csv прогнать BM25-поиск по candidates.csv и вернуть top-15 кандидатов.

Файл:

retrieval_bm25.csv

Обязательные колонки

- query_id
- candidate_id
- search_system
- rank
- score
- latency_ms
- retrieved_at

Значение search_system = "bm25"

3. Антон - Для каждого запроса из queries.csv прогнать vector retrieval по candidates.csv и вернуть top-15 кандидатов.

Файл:

retrieval_vector.csv

Обязательные колонки

- query_id
- candidate_id
- search_system
- rank

- score
- latency_ms
- retrieved_at

Значение search_system = "vector"

Входные данные для всех одинаковые:

- queries.csv
- candidates.csv

Нужно для каждого query_id получить top-15 кандидатов из своей retrieval-системы и сохранить результат в одинаковом формате.

Общие правила для всех

1. Использовать один и тот же candidates.csv
2. Прогонять все запросы из queries.csv
3. На каждый query_id вернуть не более 15 кандидатов
4. Одна строка в датасете = один кандидат для одного запроса
5. rank должен идти от 1 до 15
6. candidate_id должен совпадать с candidate_id из candidates.csv
7. Файл должен быть в long-format, не списками в ячейках

То есть, 160 запросов в queries * 15 = 2400 строк примерно в каждом файле

T3

Общая инфа

Цель подготовки датасета — а создать корректный evaluation set для сравнительного тестирования системы, как это описано в тезисе. Поэтому набор кандидатов для разметки нельзя формировать только из результатов SQL-HR. Если брать кандидатов только из нашей системы, датасет будет смещен в её сторону: judge будет оценивать только тех кандидатов, которых уже нашла SQL-HR, а значит:

- сравнение с baseline-системами станет нечестным;
- часть релевантных кандидатов, найденных BM25 или vector search, вообще не попадёт в разметку;
- метрики качества будут искажены.

Чтобы этого избежать, для каждого запроса формируется pooled set:

- top-15 из SQL-HR
- top-15 из BM25
- top-15 из vector search

Далее результаты объединяются, дубликаты удаляются, и именно этот объединенный набор кандидатов отправляется на judge-разметку.

Логика работы:

Этап 1. Retrieval по полному набору запросов

Подготовить 40-60 вакансий.

Для каждой вакансии придумать 3–5 реальных запросов рекрутера на естественном языке — обычные формулировки, не SQL.

Для каждого `query_id` из `queries.csv` нужно получить top-15 кандидатов из трёх систем:

- `sql_hr`
- `bm25`
- `vector`

Этап 2. Построение pooled set

Для каждого запроса:

- берём top-15 из `sql_hr`
- берём top-15 из `bm25`
- берём top-15 из `vector`
- объединяем результаты
- удаляем дубликаты по `candidate_id`

Получаем `pooled_candidates.csv`.

Этап 3. Выбор gold subset

Из полного набора запросов выбирается подмножество 30–50 запросов для judge-разметки.

Рекомендуемое целевое значение: 40 запросов.

Этап 4. Judge-разметка

Для каждого кандидата из pooled set по запросам из gold subset judge выставляет релевантность:

- 0 — не подходит
- 1 — частично подходит
- 2 — сильный матч

Судье передается запрос, короткий контекст вакансии и структурированный профиль кандидата (skills, опыт, роли), без полного текста резюме. Судья ставит оценку релевантности: 0 — не подходит, 1 — частично, 2 — сильный матч.

На выходе должны получиться файлы с вакансиями, запросами, pooled кандидатами и разметкой judge. Итоговый объем примерно 150–250 запросов, по 30–45 кандидатов на запрос.

Разметка judge делается только для gold subset.

Весь набор запросов нужен, чтобы гонять retrieval и мерить latency/стабильность, а gold subset — чтобы считать точные quality-метрики (nDCG, Recall и тд).

Чтобы сделать gold subset для query надо стратифицировать по категориям запросов:

1. По доменам
2. По стилю

query_type:

- strict (жёсткие фильтры)
- normal (реалистичный HR запрос)
- fuzzy (разговорный / неполный)
- complex (много условий)

domain (пример):

backend / data / qa / analyst / etc

Если gold subset = 40 queries, то берём примерно:

strict → 8
normal → 16
fuzzy → 8
complex → 8

proportional stratified sampling

Нужно минимум по несколько запросов из каждого домена.

Выбери gold subset так, чтобы:

- все типы запросов были представлены
- все домены были представлены
- длина запросов была разная (короткие/длинные)
- были и лёгкие и сложные кейсы

Этап 5. Подсчёт метрик

После judge-разметки считаются:

- Precision@5
- Precision@10
- Recall@5
- Recall@10
- Recall@15
- nDCG@5
- nDCG@10

А по полному набору запросов дополнительно считаются:

- средняя latency
- median latency
- p95 latency
- стабильность retrieval

Итоговая структура файлов

Уже готовы

candidates.csv

- candidate_id
- sex
- age
- expected_salary_rub
- desired_position
- city
- ready_to_relocate
- ready_for_business_trips
- employment_type
- work_schedule
- work_experience
- last_company
- last_job_title
- education_level_and_university
- resume_updated_at
- has_car

vacancies.csv

- vacancy_id
- job_title
- domain
- city
- salary
- salary_currency
- salary_period
- employment
- work_format
- work_hours
- work_schedule
- experience
- requirements
- responsibilities

queries.csv

- query_id
- vacancy_id
- job_title
- domain
- query_type
- query_text

Текущий объём:

- 40 вакансий
- 160 запросов
(4 запроса на каждую вакансию: *strict, normal, fuzzy, complex*)

Нужно подготовить

retrieval_results.csv

- query_id
- candidate_id
- search_system
- rank
- score
- latency_ms
- retrieved_at

pooled_candidates.csv

- query_id
- candidate_id
- from_sql_hr
- from_bm25
- from_vector
- best_rank

- num_systems_hit

gold_subset.csv

- query_id
- vacancy_id
- domain
- query_type
- query_length_bucket
- is_gold
- sampling_comment

judgments.csv

- query_id
- candidate_id
- relevance_score
- judge_model
- judge_prompt_version
- judge_comment

metrics_summary.csv

- search_system
- precision_at_5
- precision_at_10
- recall_at_5
- recall_at_10
- recall_at_15
- ndcg_at_5
- ndcg_at_10
- mean_latency_ms
- median_latency_ms
- p95_latency_ms